TASKFLOW

TASK MANAGEMENT WEB APPLICATION

MINOR PROJECT REPORT

Submitted by	Nikhat Siddekee
Course	B.Tech in Computer Science and Engineering
Project Guide	Pratham Saxena
Academic Year	2026

Developed using HTML5, CSS3, JavaScript and Browser Local Storage

CERTIFICATE

This is to certify that Nikhat Siddekee has completed the minor project titled “TaskFlow: Task Management Web Application” under the guidance of Pratham Saxena. This report describes the project implementation and its documented features.

Project Guide: Pratham Saxena
Student: Nikhat Siddekee
Academic Year: 2026

ACKNOWLEDGEMENT

I would like to thank my project guide, Pratham Saxena, for the guidance and feedback provided during the development of this project. I also thank everyone who supported me while planning, developing, and documenting TaskFlow.

ABSTRACT

TaskFlow is a browser-based task management application developed using HTML, CSS, and JavaScript. It helps users create and organize tasks, assign priorities and due dates, update task details, and track whether work is pending or complete. The application includes a dashboard, search, status and priority filters, sorting options, and visual highlighting for overdue pending tasks. Browser Local Storage is used to retain tasks after the page is refreshed.

1. INTRODUCTION

Managing assignments and everyday work can be difficult when tasks, deadlines, and priorities are not organized. TaskFlow provides a single browser interface to record tasks and track progress.

1.1 Problem Statement

Without an organized task list, users may forget deadlines or have difficulty identifying which tasks need attention first. TaskFlow addresses this by displaying task details, status, priority, and due dates together.

1.2 Project Objective

The objective is to develop a responsive task management application that enables users to create, view, edit, delete, search, sort, filter, and track tasks through a clear interface.

2. TECHNOLOGY STACK

HTML5: page structure, dashboard, form, and task list.
CSS3: visual styling and responsive layouts for desktop, tablet, and mobile screens.
JavaScript: task operations, validation, search, filters, sorting, dashboard statistics, and DOM updates.
Browser Local Storage: persistence of task data in the current browser.
Git and GitHub: source-code version control and repository hosting.

3. APPLICATION FEATURES

3.1 Task Management

Users can create tasks with a title, description, priority, and due date. Existing tasks can be edited or deleted. A confirmation prompt is shown before deletion.

3.2 Completion Status and Dashboard

Tasks can be marked completed or pending. The dashboard displays total, pending, completed, and overdue task counts. Dashboard statistics are recalculated when the task list changes.

3.3 Search, Filters, and Sorting

Users can search by task title or description, filter by completion status and priority, and sort by due date, priority, or creation time.

3.4 Overdue Indicator and Validation

A pending task with a due date earlier than the current date is identified as overdue and receives a visible label and highlighted card. Required fields and priority values are checked before a task is saved.

3.5 Data Persistence

The task array is converted to JSON and stored in Local Storage. When the application loads, saved data is read and parsed back into JavaScript objects. Data remains in the same browser profile and is not automatically synchronized across devices.

4. APPLICATION WORKFLOW

1. The user opens index.html in a web browser.
2. The user enters a title, description, priority, and due date.
3. JavaScript validates the form and adds or updates a task.
4. The task is rendered in the task list.
5. The user can search, filter, sort, complete, edit, or delete tasks.
6. Dashboard counts update and task data is saved to Local Storage.
7. On a later page load or refresh, saved tasks are restored.

5. USER INTERFACE AND IMPLEMENTATION

The interface contains a header, dashboard cards, task form, search field, filter and sort controls, and task cards. JavaScript uses reusable helper functions to create elements, load and save tasks, display filtered tasks, and update dashboard statistics. CSS media queries adapt the dashboard, form, filters, and action buttons to smaller screens.

6. SCREENSHOTS OF THE APPLICATION

The following figures are screenshots supplied with the project and show the implemented interface.

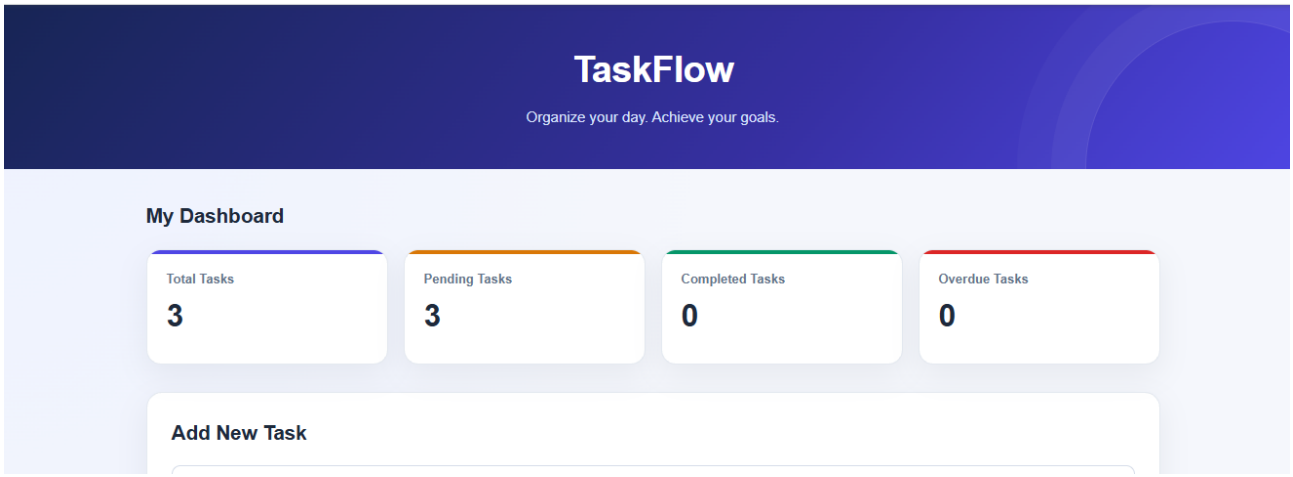


Figure 1. Main dashboard with task summary cards.

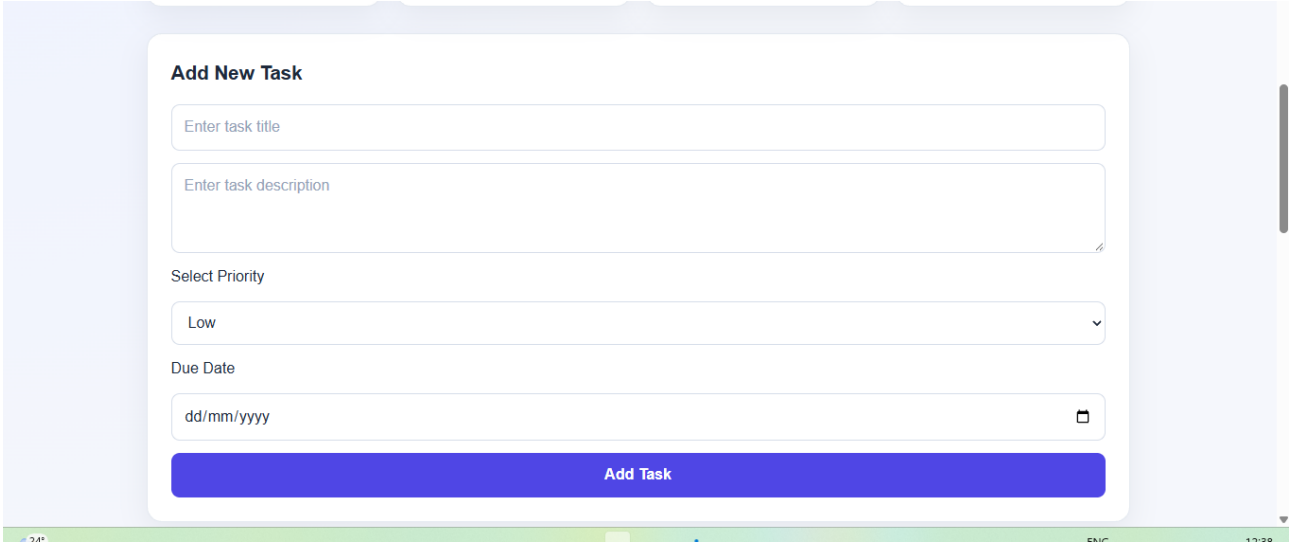


Figure 2. Task creation form with title, description, priority and due date.

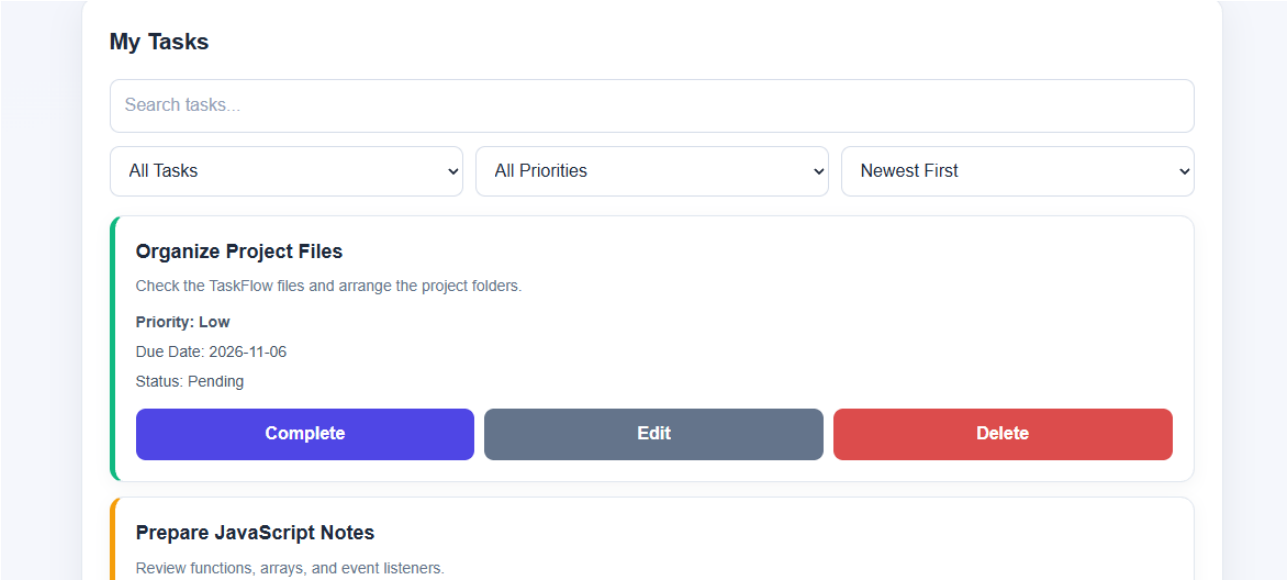


Figure 3. Task list with search, filters, sorting, status and task actions.

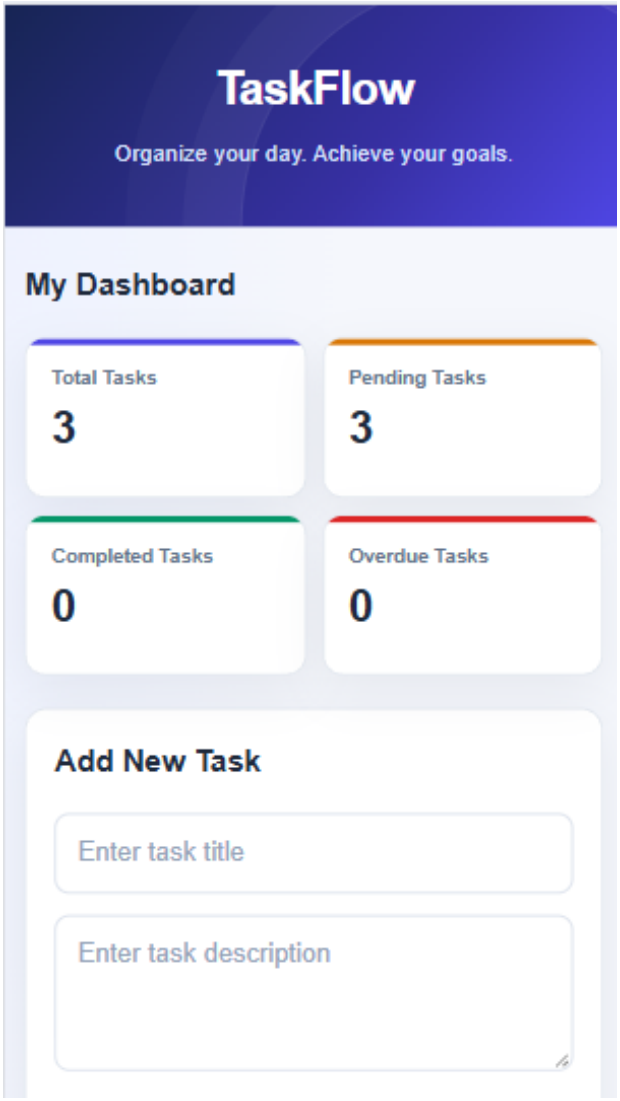


Figure 4. Responsive mobile layout.

7. TESTING CHECKLIST

Use these cases during the live demonstration. Record the actual result after running each test in the target browser; the report does not claim unverified test execution.

Test case	Expected result	Result
Add task with valid details	Task appears in the list; total and pending counts update.	To verify
Submit missing required fields	Incomplete task is not added.	To verify
Edit a task	Updated details appear in the task card.	To verify
Delete a task	Confirmation is requested before deletion.	To verify
Complete / mark pending	Status and dashboard counts update.	To verify
Search and filter	Only matching tasks are displayed.	To verify
Sort tasks	Tasks are ordered by selected criterion.	To verify
Refresh browser	Previously saved tasks are restored.	To verify

8. CHALLENGES AND SOLUTIONS

A key implementation concern is keeping the dashboard, task cards, and browser storage synchronized. The application centralizes rendering and dashboard updates so that task changes are reflected consistently. Another concern is supporting smaller screens; CSS grid, flexible controls, and media queries are used to adapt the layout.

9. LIMITATIONS AND FUTURE ENHANCEMENTS

The current application stores data locally in the browser and does not provide user accounts, cloud synchronization, or multi-device access. Future enhancements could include optional user authentication, cloud backup, recurring tasks, reminders, and export of task data.

10. CONCLUSION

TaskFlow demonstrates how HTML, CSS, JavaScript, and browser Local Storage can be combined to build a responsive task management application. It provides core task operations, dashboard statistics, search, filtering, sorting, overdue indicators, and persistent browser storage.

11. REFERENCES

• MDN Web Docs: HTML, CSS, JavaScript, and Web Storage API documentation.
• Project implementation files included in the source_code folder.